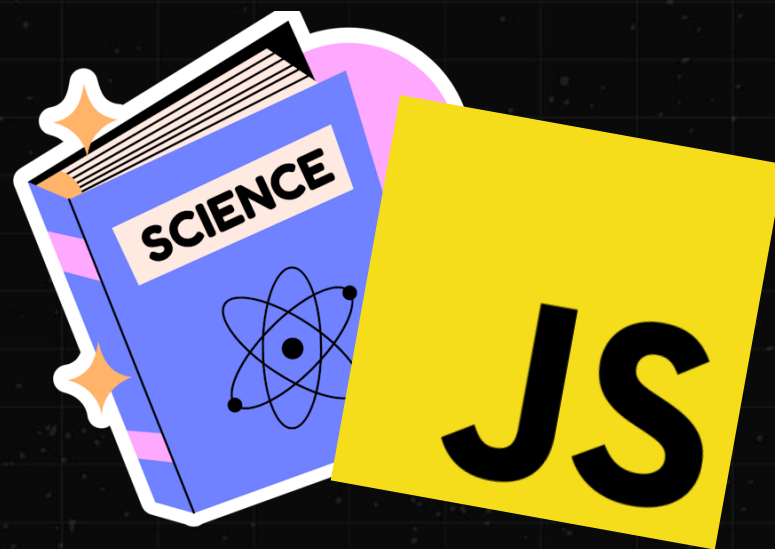


ЛЕКЦИЯ #22

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Продолжаем изучать регулярные выражения
- * Изучаем **возможности опережающей и ретроспективной проверок**
- * Разбираем множество возможных **флагов компиляции регулярных выражений в JS**
- * Обсуждаем нюансы

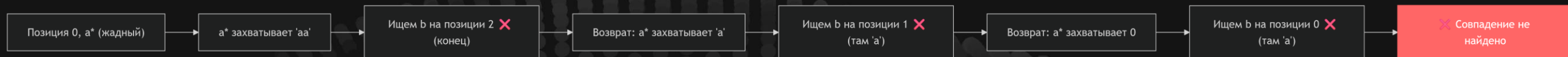
«ДВИЖЕНИЕ» ПО СТРОКЕ

- * Когда КА, построенный на основе регулярного выражения, находит совпадение по символу в тексте, он **перемещает позицию поиска на один символ**
- * Таким образом, автомат как бы «движется» по строке
- * Если следующий символ не подходит под шаблон, автомат **делает возврат и пробует другой вариант**

ТАКЖЕ НУЖНО ПОНИМАТЬ

- * Поиск в строке **всегда начинается с некоторой позиции**
- * На каждой итерации конечный автомат **потребляет ровно один символ**
- * То есть мы **сдвигаемся каждый раз** на один символ вперёд

"aaaa".match(/a*b/)



А МОЖНО ЛИ ПОДГЛЯДЕТЬ
ВПЕРЕД БЕЗ СМЕЩЕНИЯ
ПОЗИЦИИ ПОИСКА?



МОЖНО

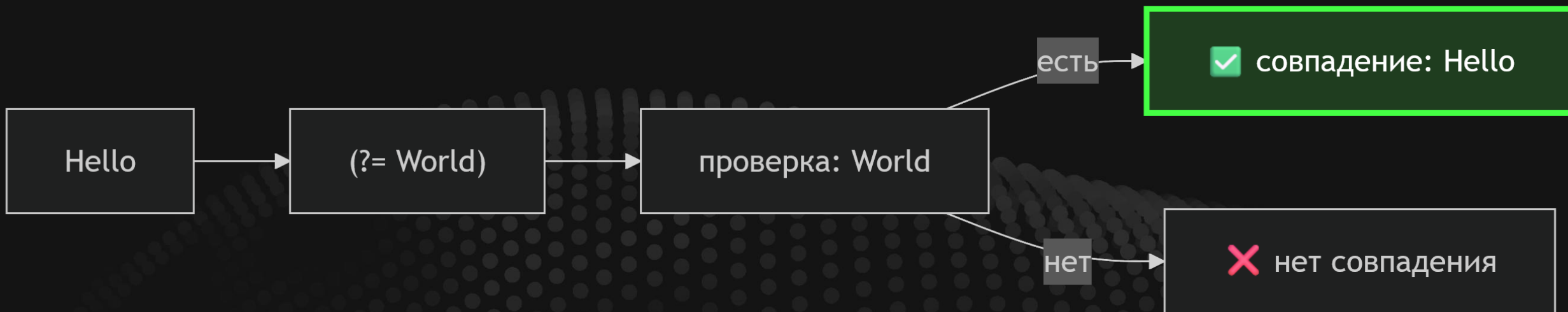
- * Это делается с помощью опережающих проверок (lookahead)
- * Они бывают двух видов: позитивные и негативные

```
console.log(/Hello(?: World)/.exec("Hello World")); // [ 'Hello' ]  
console.log(/Hello(?: World)/.exec("Hello Bob!")); // null  
console.log(/Hello(?:! World)/.exec("Hello Bob!")); // [ 'Hello' ]
```

РЕФЛЕКСИРУЕМ

- * `/Hello(=? World)/` – ищем «Hello» только если потом идет « World»
- * `/Hello(?! World)/` – ищем «Hello» только если потом не идет « World»
- * «Подглядывание» (lookaround) **не сдвигает позицию автомата** и не фигурирует в конечном результате

ВИЗУАЛИЗИРУЕМ



А ТАКИЕ СКОБКИ
СОЗДАЮТ ГРУППУ?



НЕТ

- * Но мы можем создать её явно, например, `/Hello(?=( World))/`
- * И даже сделать на неё обратную ссылку
- * `/Hello(?=( World))\1/.exec("Hello World") // ['Hello World', ' World']`
- * Несмотря на кажущуюся бессмысленность, этот подход **позволяет оптимизировать некоторые выражения**

«АТОМАРНАЯ ГРУППА»

- * Используя опережающую проверку и обратную ссылку, мы можем **сократить количество возвратов в случае неудачи до одного**
- * **Это крайне важно** для избежания катастрофических возвратов

```
console.log(/^(?=(\d+))\1*$/ .exec("11111111111111111111111111111111!")); // Моментально
console.log(/^(\\d+)*$/ .exec("11111111111111111111111111111111!")); // Зависает
```

РЕФЛЕКСИРУЕМ

- * При опережающей проверке **позиция поиска остаётся неизменной**
- * Поэтому `/Hello(=? World) World/` — это то же самое, что и просто `/Hello World/`

А **МОЖНО ЛИ** ИСПОЛЬЗОВАТЬ
ОПЕРЕЖАЮЩИЕ ПРОВЕРКИ
С КВАНТИФИКАТОРАМИ?



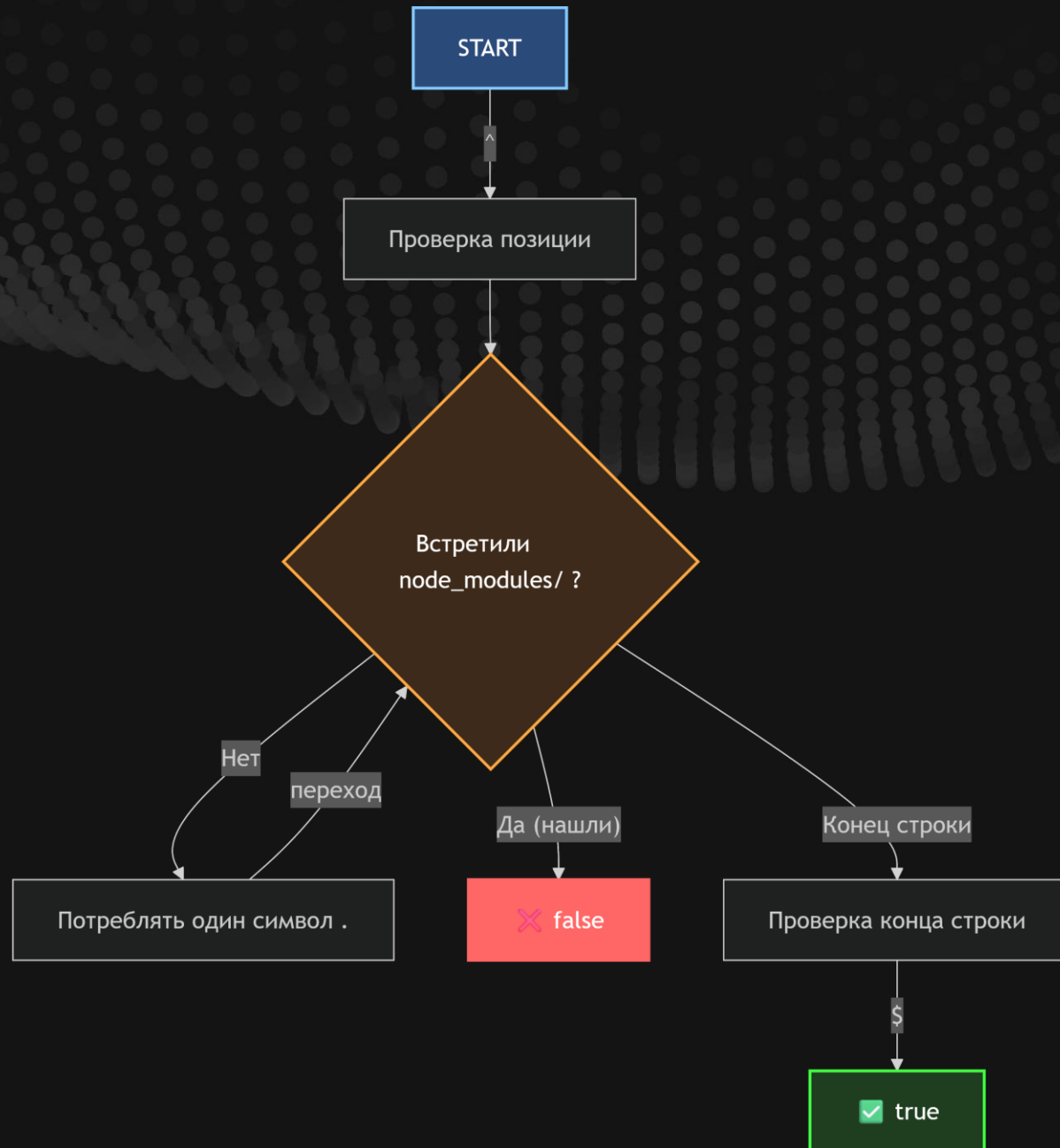
БЕЗУСЛОВНО

- ✱ Например, напомним RegExr, который дает true только если в строке нет фрагмента `/node_modules/`



```
const isLocalPath = /^(?!\/node_modules\/).)*$/;

console.log(isLocalPath.test("/foo/bar/bla"));           // true
console.log(isLocalPath.test("/node_modules/foo/bar/bla")); // false
console.log(isLocalPath.test("/foo/node_modules/bar/bla")); // false
```

РЕФЛЕКСИРУЕМ

- * Мы создали цикл, где перед каждым символом **проверяется отсутствие некоторой подстроки впереди**
- * Это удобно использовать для выражения вида: **любая строка, кроме этой**

А МОЖНО ЛИ
СМОТРЕТЬ
НАЗАД?



РЕТРОСПЕКТИВНЫЕ ПРОВЕРКИ



```
console.log(/(?<=Hello )World/.exec("Hello World")); // [ 'World' ]  
console.log(/(?<=Hello )World/.exec("Bye World")); // null  
console.log(/(?<!=Hello )World/.exec("Bye World")); // [ 'World' ]
```


ТУТ ТАКОЙ ЖЕ ПРИНЦИП

- * `/(?<=Hello)World/` – ищем «World» только если перед идет «Hello »
- * `/(?<!Hello )World/` – ищем «World» только если перед не идет «Hello »
- * Можно **совмещать ретроспективные и опережающие проверки** в одном выражении
- * `console.log(/(?<=("["])).*?(?=\1)/.exec("Hello 'World!'")); // [ 'World', "" ]`

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ГЛУБОКИЙ ВДОХ – ВЫДОХ

- ✱ Опережающие проверки – это самое сложное, что есть в регулярных выражениях
- ✱ Здесь нужна **практика, и только практика**

А МОЖНО ЛИ УЗНАТЬ
ПОЗИЦИЮ ПОИСКА REGEXR?



ДА

- * У объекта `RegExp` есть свойство `lastIndex`
- * Оно определяет **позицию в строке, с которой начинается поиск**
- * **Это свойство можно менять**, если у регулярного выражения есть флаг `g` (глобальный поиск)



```
const regex = /\d+/g;
```

```
const str = "12-34-56-78";
```

```
console.log(regex.exec(str)); // [ '12' ]
```

```
console.log(regex.lastIndex); // 2
```

```
console.log(regex.exec(str)); // [ '34' ]
```

```
console.log(regex.lastIndex); // 5
```

```
regex.lastIndex = 2;
```

```
console.log(regex.exec(str)); // [ '34' ]
```

```
console.log(regex.lastIndex); // 5
```

```
regex.lastIndex = 20;
```

```
console.log(regex.exec(str)); // null
```

```
console.log(regex.lastIndex); // 0
```

А ЧТО ЕЩЕ ЗА
ФЛАГИ ТАКИЕ?



ФЛАГИ КОМПИЛЯЦИИ

- * Одно и то же выражение **можно интерпретировать по-разному** в зависимости от заданных параметров
- * Флаги задаются после закрывающего `/` или вторым параметром в `RegExp`
- * Можно задать **сразу несколько флагов** (порядок не важен)
- * `/^[a-z]+/gim` или `new RegExp("^[a-z]+", "gim")`
- * С помощью свойства `flags` можно узнать, **какие флаги есть у конкретного регулярного выражения**
- * `console.log(/^[a-z]+/gim.flags) // gim`

g

- * Флаг **g** влияет на работу с регулярным выражением
- * **Можно менять свойство `lastIndex`** для управления позицией поиска
- * После успешного нахождения подстроки поиск продолжится с того места, **где закончилось предыдущее совпадение**
- * И так до тех пор, **пока не будет достигнут конец строки**
- * Свойство **`global`** показывает, **задан ли такой флаг** у конкретного регулярного выражения



```
const regex = /ab/g;  
const text = "ab ab ab";  
  
// Проверяем флаг  
console.log(regex.global); // true  
console.log(regex.flags); // "g"  
  
console.log(regex.test(text)); // true  
console.log(regex.lastIndex); // 2  
  
console.log(regex.test(text)); // true  
console.log(regex.lastIndex); // 5  
  
console.log(regex.test(text)); // true  
console.log(regex.lastIndex); // 8  
  
console.log(regex.test(text)); // false  
console.log(regex.lastIndex); // 0 (сброс)
```

ОБРАТИТЕ ВНИМАНИЕ

- * Метод `test` также учитывает `lastIndex` при наличии флага `g`
- * Это может привести к сложно отлавливаемым ошибкам
- * Метод `match` меняет свой формат ответа, если задан флаг `g` – он **возвращает массив всех вхождений без запоминающихся групп**



```
const text = "ab ab ab";
```

```
// Без флага g — возвращает первое совпадение + группы
```

```
const match1 = text.match(/ab/);
```

```
console.log(match1); // [ 'ab', index: 0, input: 'ab ab ab', groups: undefined ]
```

```
// С флагом g — возвращает массив всех совпадений (без групп)
```

```
const match2 = text.match(/ab/g);
```

```
console.log(match2); // [ 'ab', 'ab', 'ab' ]
```

replace И g

- * Метод строки `replace` всегда выполняет замену, начиная с начала строки (не учитывая `lastIndex`)
- * Таким образом, `lastIndex` после `replace` всегда будет равен 0
- * Такое же поведение у `replaceAll`



```
const regex = /ab/g;  
const text = "ab ab ab";
```

```
// Устанавливаем lastIndex  
regex.lastIndex = 4;  
console.log(regex.lastIndex); // 4
```

```
// replace не учитывает lastIndex и начинает с начала  
const result = text.replace(regex, "X");  
console.log(result); // "X X X"  
console.log(regex.lastIndex); // 0 (сброс)
```

```
// replaceAll — аналогично  
regex.lastIndex = 4;  
const result2 = text.replaceAll(regex, "X");  
console.log(result2); // "X X X"  
console.log(regex.lastIndex); // 0
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Флаг `g` может быть достаточно коварным при неаккуратном использовании
- * Если вы хотите сделать глобальный поиск, то лучше использовать методы `matchAll` и `replaceAll`
- * Особенность этих методов в том, что если им передать регулярное выражение без флага `g`, будет выброшено исключение
- * К тому же `matchAll` всегда возвращает итератор, что удобнее формата массива



```
const text = "ab ab ab";
```

```
// ❌ Старый подход (с флагом g и match)
```

```
const oldWay = text.match(/ab/g);
```

```
console.log(oldWay); // [ 'ab', 'ab', 'ab' ] (массив, без групп)
```

```
// ✅ Современный подход (matchAll)
```

```
const matchAllIterator = text.matchAll(/ab/g);
```

```
for (const match of matchAllIterator) {
```

```
  console.log(match);
```

```
  // [ 'ab', index: 0, input: 'ab ab ab', groups: undefined ]
```

```
  // [ 'ab', index: 3, input: 'ab ab ab', groups: undefined ]
```

```
  // [ 'ab', index: 6, input: 'ab ab ab', groups: undefined ]
```

```
}
```

```
// ❌ Ошибка: без флага g
```

```
try {
```

```
  const wrong = text.matchAll(/ab/);
```

```
} catch (err) {
```

```
  console.error(err.message); // String.prototype.matchAll called with a non-global RegExp argument
```

```
}
```

i

- * Флаг i означает, что следует **не различать верхний и нижний регистр символов**
- * Свойство ignoreCase показывает, задан ли такой флаг у конкретного выражения



```
// Без флага i — регистр важен
const regex1 = /hello/;
console.log(regex1.test("Hello")); // false

// С флагом i — регистр игнорируется
const regex2 = /hello/i;
console.log(regex2.test("Hello")); // true
console.log(regex2.test("HELLO")); // true
console.log(regex2.test("hElLo")); // true

// Проверка свойства ignoreCase
console.log(regex2.ignoreCase); // true
console.log(regex1.ignoreCase); // false
```


m

- * Флаг **m** (multiline) влияет на якорные ссылки **^** и **\$**
- * Начало и конец строки **считаются от каждой новой строки, а не от всего текста**
- * Свойство **multiline** показывает, задан ли такой флаг у конкретного выражения



```
const text = "Hello World\nHello JavaScript\nHello React";

// Без флага m — ^ и $ работают только для всей строки
const regex1 = /^Hello/;
console.log(text.match(regex1)); // [ 'Hello' ] (только первая строка)

// С флагом m — ^ и $ работают для каждой строки
const regex2 = /^Hello/gm;
console.log(text.match(regex2)); // [ 'Hello', 'Hello', 'Hello' ]

// $ с флагом m
const regex3 = /React$/gm;
console.log(text.match(regex3)); // [ 'React' ] (последняя строка)

// Проверка свойства multiline
console.log(regex2.multiline); // true
console.log(regex1.multiline); // false
```

u

- * Флаг `u` (unicode) включает **режим расширенной поддержки Юникода**
- * Позволяет использовать диапазоны символов, **которые кодируются суррогатными парами**, например [😄 - 😱]
- * Появляется возможность использовать новый символьный класс `\p{}`
- * Свойство `unicode` показывает, задан ли такой флаг у конкретного выражения



```
// ❌ Без флага u — синтаксическая ошибка  
// const regex1 = /[😄-😱]/;
```

```
// ✅ С флагом u — все работает  
const regex2 = /[😄-😱]/u;  
console.log(regex2.test("😄")); // true  
console.log(regex2.test("😁")); // true  
console.log(regex2.test("😊")); // false
```

```
// Проверка свойства unicode  
console.log(regex2.unicode); // true
```




```
// ✅ Найти все буквы (любого алфавита)
const letters = /\p{L}+/gu;
console.log("Hello Привет 你好".match(letters));
// [ 'Hello', 'Привет', '你好' ]
```

```
// ✅ Найти все кириллические буквы
const cyrillic = /\p{Script=Cyrillic}+/gu;
console.log("Hello Привет 你好".match(cyrillic));
// [ 'Привет' ]
```

```
// ✅ Найти все эмодзи
const emoji = /\p{Emoji}/gu;
console.log("Привет 😄 и 😁".match(emoji));
// [ '😄', '😁' ]
```

```
// ✅ Найти все числа (включая римские)
const numbers = /\p{N}/gu;
console.log("123 IV V".match(numbers));
// [ '1', '2', '3', 'IV', 'V' ]
```

y

- * Флаг **y** (sticky) включает «липкий» режим работы регулярного выражения
- * Как и при глобальном поиске, такое выражение может искать в строке несколько раз
- * Но **после первого «неуспеха» поиск сбрасывается**
- * Можно совмещать с флагом **g**
- * Свойство **sticky** показывает, задан ли такой флаг у конкретного выражения



```
const text = "ab ab ab";

// С флагом y
const regex = /ab/y;

console.log(regex.exec(text)); // [ 'ab' ] (на позиции 0)
console.log(regex.lastIndex); // 2

console.log(regex.exec(text)); // null (потому что на позиции 2 пробел, а не "ab")
console.log(regex.lastIndex); // 0 (сброс)

// С флагом g
const regex2 = /ab/g;

console.log(text.match(regex2)); // [ 'ab', 'ab', 'ab' ]

// С флагом gy
const regex3 = /ab/gy;

console.log(text.match(regex3)); // [ 'ab' ]
```



```
const text = "ab ab ab";

// С флагом y
const regex = /ab/y;
regex.lastIndex = 3; // Не игнорируется

console.log(text.replace(regex, "XX")); // ab XX ab

// С флагом g
const regex2 = /ab/g;
regex2.lastIndex = 3; // Игнорируется

console.log(text.replace(regex2, "XX")); // XX XX XX

// С флагом gy
const regex3 = /ab/gy;
regex3.lastIndex = 3; // Игнорируется

console.log(text.replace(regex3, "XX")); // XX ab ab
```


S

- * Флаг `s` (`dotAll`) включает режим, в котором точка `.` соответствует любому символу, включая перевод строки
- * Свойство `dotAll` показывает, задан ли такой флаг у конкретного выражения



```
const text = "Hello\nWorld";
```

```
// ❌ Без флага s — точка не захватывает перевод строки
```

```
const regex1 = /Hello.World/;
```

```
console.log(regex1.test(text)); // false
```

```
// ✅ С флагом s — точка захватывает перевод строки
```

```
const regex2 = /Hello.World/s;
```

```
console.log(regex2.test(text)); // true
```

```
// Проверка свойства dotAll
```

```
console.log(regex2.dotAll); // true
```

```
console.log(regex1.dotAll); // false
```

v

- * Флаг `v` (`unicodeSets`) – схожий с `u` режим, добавляющий поддержку операций над множествами Юникода
- * Например, `/[\p{Script_Extensions=Greek}&&\p{Letter}]/v`
- * Или `/^[ \p{Script_Extensions=Greek}--\q{\alpha} ]$/v`
- * Свойство `unicodeSets` показывает, задан ли такой флаг у конкретного выражения



```
// 1. Пересечение множеств: буквы греческого алфавита
const greek = /[\p{Script_Extensions=Greek}&&\p{Letter}]/v;
console.log(greek.test("α")); // true (греческая буква)
console.log(greek.test("A")); // false (латиница)

// 2. Вычитание множеств: все греческие буквы, кроме альфа
const regex = /^[\p{Script_Extensions=Greek}--\p{α}]/v;

console.log(regex.test("β")); // true (бета — не исключена)
console.log(regex.test("γ")); // true (гамма — не исключена)
console.log(regex.test("α")); // false (исключена)

// 3. Проверка свойства unicodeSets
console.log(greek.unicodeSets); // true
```


second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Поначалу кажется, что это невозможно запомнить
- * Но просто нужна практика 😊

ЕЩЕ ПОЛЕЗНЫЕ СВОЙСТВА

- * У регулярного выражения есть свойство `source`
- * Его удобно использовать для создания **новых регулярных выражений на основе текущего**
- * У конструктора `RegExp` есть **полезный набор статических свойств**, которые относятся к работе последнего регулярного выражения
- * Хотя этот API считается устаревшим (deprecated)



```
const regex1 = /[a-z]+/g;  
console.log(regex1.source); // "[a-z]+"
```

```
// Создаём новое выражение на основе существующего  
const regex2 = new RegExp(regex1.source, "i");  
console.log(regex2); // /[a-z]+/i
```



```
// ❌ Устаревший подход
const text = "Hello World";
const regex = /(World)/;
regex.test(text);

console.log(RegExp.input);    // "Hello World"
console.log(RegExp.lastMatch); // "World"
console.log(RegExp.$1);       // "World"

// ✅ Современный подход
const match = regex.exec(text);
console.log(match.input); // "Hello World"
console.log(match[0]);    // "World"
```

НЮАНСИКИ

- * Все методы строк, такие как `replace`, `match`, `search` и другие, могут принимать не только `RegExp`, но и строки
- * Некоторые методы, например, `search`, неявно создают `RegExp` из строки
- * Это может приводить к сложно отлавливаемым багам
- * Лучше явно передавайте `RegExp`



// ❌ Неявное создание RegExp из строки

```
const text = "Hello. World.";
const result = text.search(".");
console.log(result); // 0 (не 5!)
```

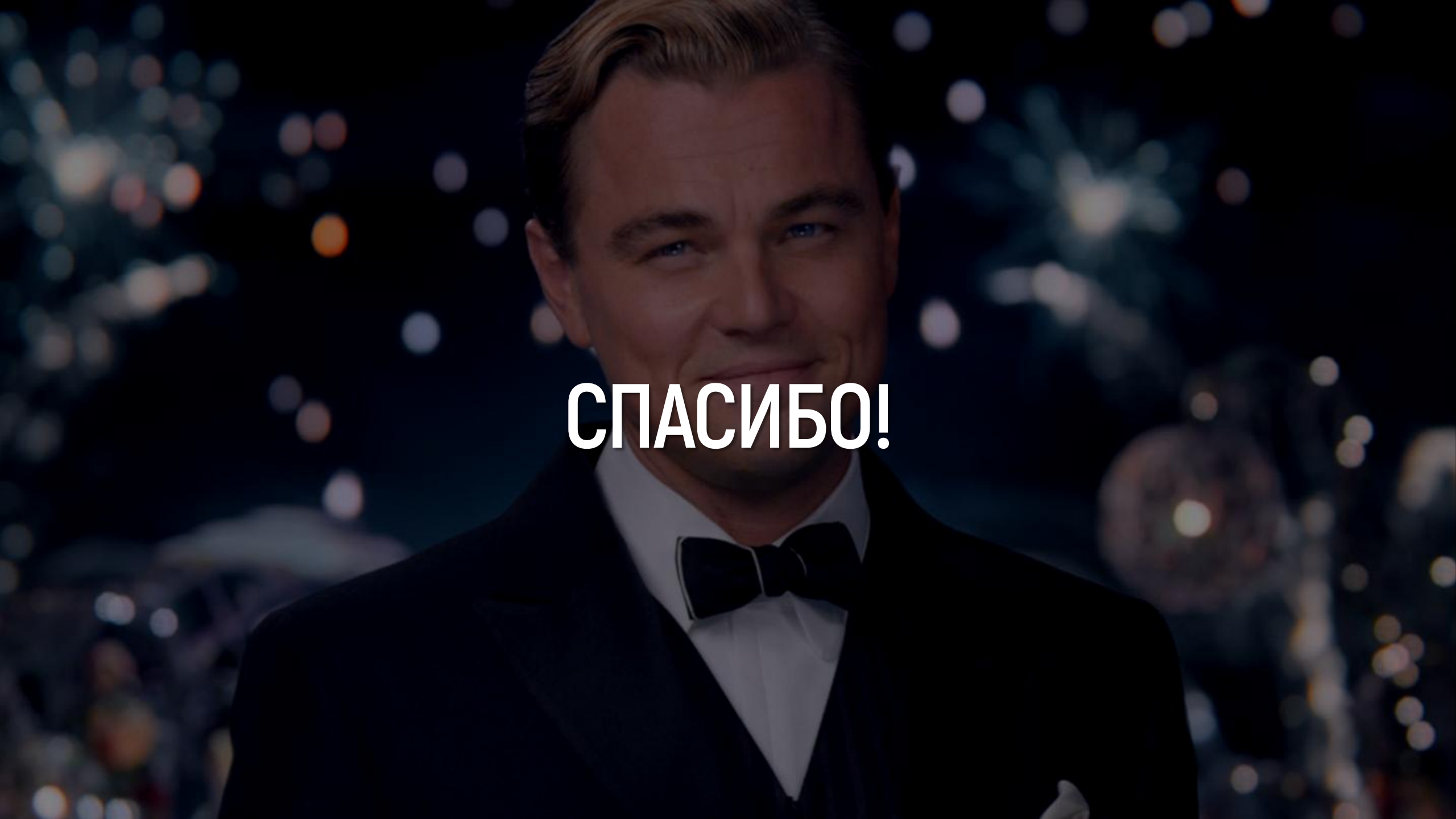
// Почему? ".".search(".") ищет первый символ, а не точку как литерал!

// ✅ Явное создание RegExp

```
const result2 = text.search(/\./);
console.log(result2); // 5 (точка после Hello)
```

ПОДВЕДЁМ ИТОГИ

- * Регулярные выражения — это отдельный невероятно полезный DSL для работы со строками
- * Регулярки присутствуют в большинстве языков программирования, хотя их синтаксис может отличаться
- * Также регулярки можно использовать в подавляющем большинстве инструментов для работы с текстом и файловой системой
- * Любой, кто хочет называть себя Middle, обязан хотя бы на среднем уровне владеть регулярками
- * Лучшие там, где сложно

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus, colorful bokeh lights, suggesting a night-time event or fireworks.

СПАСИБО!